

(How to Write a (Lisp) Interpreter (in Python))

This page has two purposes: to describe how to implement computer language interpreters in general, and in particular to build an interpreter for most of the [Scheme](#) dialect of Lisp using [Python 3](#) as the implementation language. I call my language and interpreter *Lispy* ([lis.py](#)). Years ago, I showed how to write a semi-practical Scheme interpreter [Java](#) and in [in Common Lisp](#)). This time around the goal is to demonstrate, as concisely and simply as possible, what [Alan Kay](#) called "[Maxwell's Equations of Software](#)."

Why does this matter? As [Steve Yegge said](#), "*If you don't know how compilers work, then you don't know how computers work.*" Yegge describes 8 problems that can be solved with compilers (or equally well with interpreters, or with Yegge's typical heavy dosage of cynicism).

Syntax and Semantics of Scheme Programs

The *syntax* of a language is the arrangement of characters to form correct statements or expressions; the *semantics* is the meaning of those statements or expressions. For example, in the language of mathematical expressions (and in many programming languages), the syntax for adding one plus two is " $1 + 2$ " and the semantics is the application of the addition operation to the two numbers, yielding the value 3. We say we are *evaluating* an expression when we determine its value; we would say that " $1 + 2$ " evaluates to 3, and write that as " $1 + 2 \Rightarrow 3$ ".

Scheme syntax is different from most other programming languages. Consider:

Java	Scheme
<pre>if (x.val() > 0) { return fn(A[i] + 3 * i, new String[] {"one", "two"}); }</pre>	<pre>(if (> (val x) 0) (fn (+ (aref A i) (* 3 i)) (quote (one two))))</pre>

Java has a wide variety of syntactic conventions (keywords, infix operators, three kinds of brackets, operator precedence, dot notation, quotes, commas, semicolons), but Scheme syntax is much simpler:

- Scheme programs consist solely of *expressions*. There is no statement/expression distinction.
- Numbers (e.g. 1) and symbols (e.g. A) are called *atomic expressions*; they cannot be broken into pieces. These are similar to their Java counterparts, except that in Scheme, operators such as + and > are symbols too, and are treated the same way as A and fn.
- Everything else is a *list expression*: a "(", followed by zero or more expressions, followed by a ")". The first element of the list determines what it means:
 - A list starting with a keyword, e.g. (if ...), is a *special form*; the meaning depends on the keyword.
 - A list starting with a non-keyword, e.g. (fn ...), is a function call.

The beauty of Scheme is that the full language only needs 5 keywords and 8 syntactic forms. In comparison, Python has 33 keywords and [110](#) syntactic forms, and Java has 50 keywords and [133](#) syntactic forms. All those parentheses may seem intimidating, but Scheme syntax has the virtues of simplicity and consistency. (Some have joked that "Lisp" stands for "[Lots of Irritating Silly Parentheses](#)"; I think it stand for "[Lisp Is Syntactically Pure](#)".)

In this page we will cover all the important points of the Scheme language and its interpretation (omitting some minor details), but we will take two steps to get there, defining a simplified language first, before defining the near-full Scheme language.

Language 1: Lispy Calculator

Lispy Calculator is a subset of Scheme using only five syntactic forms (two atomic, two special forms, and the procedure call). Lispy Calculator lets you do any computation you could do on a typical calculator—as long as you are comfortable with prefix notation. And you can do two things that are not offered in typical calculator languages: "if" expressions, and the definition of new variables. Here's an example program, that computes the area of a circle of radius 10, using the formula πr^2 :

```
(define r 10)
(* pi (* r r))
```

Here is a table of all the allowable expressions:

Expression	Syntax	Semantics and Example
variable reference	<i>symbol</i>	A symbol is interpreted as a variable name; its value is the variable's value. Example: $r \Rightarrow 10$ (assuming r was previously defined to be 10)
constant literal	<i>number</i>	A number evaluates to itself. Examples: $12 \Rightarrow 12$ or $-3.45e+6 \Rightarrow -3.45e+6$
conditional	(if <i>test</i> <i>conseq</i> <i>alt</i>)	Evaluate <i>test</i> ; if true, evaluate and return <i>conseq</i> ; otherwise <i>alt</i> . Example: (if (> 10 20) (+ 1 1) (+ 3 3)) $\Rightarrow 6$
definition	(define <i>symbol</i> <i>exp</i>)	Define a new variable and give it the value of evaluating the expression <i>exp</i> . Examples: (define r 10)
procedure call	(<i>proc</i> <i>arg</i> ...)	If <i>proc</i> is anything other than one of the symbols if, define, or quote then it is treated as a procedure. Evaluate <i>proc</i> and all the <i>args</i> , and then the procedure is applied to the list of <i>arg</i> values. Example: (sqrt (* 2 8)) $\Rightarrow 4.0$

In the Syntax column of this table, *symbol* must be a symbol, *number* must be an integer or floating point number, and the other italicized words can be any expression. The notation *arg*... means zero or more repetitions of *arg*.

What A Language Interpreter Does

A language interpreter has two parts:

1. **Parsing:** The parsing component takes an input program in the form of a sequence of characters, verifies it according to the *syntactic rules* of the language, and translates the program into an internal representation. In a simple interpreter the internal representation is a tree structure (often called an *abstract syntax tree*) that closely mirrors the nested structure of statements or expressions in the program. In a language translator called a *compiler* there is often a series of internal representations, starting with an abstract syntax tree, and progressing to a sequence of instructions that can be directly executed by the computer. The Lispy parser is implemented with the function `parse`.
2. **Execution:** The internal representation is then processed according to the *semantic rules* of the language, thereby carrying out the computation. Lispy's execution function is called `eval` (note this shadows Python's built-in function of the same name).

Here is a picture of the interpretation process:

program $\Rightarrow$ parse $\Rightarrow$ abstract-syntax-tree $\Rightarrow$ eval $\Rightarrow$ result

And here is a short example of what we want `parse` and `eval` to be able to do (`begin` evaluates each expression in order and returns the final one):

```
>> program = "(begin (define r 10) (* pi (* r r)))"
>>> parse(program)
['begin', ['define', 'r', 10], ['*', 'pi', ['*', 'r', 'r']]]
>>> eval(parse(program))
314.1592653589793
```

Type Definitions

Let's be explicit about our representations for Scheme objects:

```
Symbol = str          # A Scheme Symbol is implemented as a Python str
Number = (int, float) # A Scheme Number is implemented as a Python int or float
Atom    = (Symbol, Number) # A Scheme Atom is a Symbol or Number
List    = list        # A Scheme List is implemented as a Python list
Exp     = (Atom, List) # A Scheme expression is an Atom or List
Env     = dict         # A Scheme environment (defined below)
                        # is a mapping of {variable: value}
```

Parsing: parse, tokenize and read_from_tokens

Parsing is traditionally separated into two parts: *lexical analysis*, in which the input character string is broken up into a sequence of *tokens*, and *syntactic analysis*, in which the tokens are assembled into an abstract syntax tree. The Lispy tokens are parentheses, symbols, and numbers. There are many tools for lexical analysis (such as Mike Lesk and Eric Schmidt's [lex](#)), but for now we'll use a very simple tool: Python's `str.split`. The function `tokenize` takes as input a string of characters; it adds spaces around each paren, and then calls `str.split` to get a list of tokens:

```
def tokenize(chars: str) -> list:
    "Convert a string of characters into a list of tokens."
    return chars.replace('(', ' ( ').replace(')', ' ) ').split()
```

Here we apply `tokenize` to our sample program:

```
>>> program = "(begin (define r 10) (* pi (* r r)))"
>>> tokenize(program)
['(', 'begin', '(', 'define', 'r', '10', ')', '(', '*', 'pi', '(', '*', 'r', 'r', ')', ')', ')']
```

Our function `parse` will take a string representation of a program as input, call `tokenize` to get a list of tokens, and then call `read_from_tokens` to assemble an abstract syntax tree. `read_from_tokens` looks at the first token; if it is a `)` that's a syntax error. If it is a `(`, then we start building up a list of sub-expressions until we hit a matching `)`. Any non-parenthesis token must be a symbol or number. We'll let Python make the distinction between them: for each non-paren token, first try to interpret it as an int, then as a float, and if it is neither of those, it must be a symbol. Here is the parser:

```
def parse(program: str) -> Exp:
    "Read a Scheme expression from a string."
    return read_from_tokens(tokenize(program))

def read_from_tokens(tokens: list) -> Exp:
    "Read an expression from a sequence of tokens."
    if len(tokens) == 0:
        raise SyntaxError('unexpected EOF')
    token = tokens.pop(0)
    if token == '(':
        L = []
        while tokens[0] != ')':
            L.append(read_from_tokens(tokens))
            tokens.pop(0) # pop off ')'
        return L
    elif token == ')':
        raise SyntaxError('unexpected )')
    else:
        return atom(token)

def atom(token: str) -> Atom:
    "Numbers become numbers; every other token is a symbol."
    try: return int(token)
    except ValueError:
        try: return float(token)
        except ValueError:
            return Symbol(token)
```

`parse` works like this:

```
>>> program = "(begin (define r 10) (* pi (* r r)))"
>>> parse(program)
['begin', ['define', 'r', 10], ['*', 'pi', ['*', 'r', 'r']]]
```

We're almost ready to define `eval`. But we need one more concept first.

Environments

An environment is a mapping from variable names to their values. By default, `eval` will use a global environment that includes the names for a bunch of standard functions (like `sqrt` and `max`, and also operators like `*`). This environment can be augmented with user-defined variables, using the expression `(define symbol value)`.

```
import math
import operator as op

def standard_env() -> Env:
    "An environment with some Scheme standard procedures."
    env = Env()
    env.update(vars(math)) # sin, cos, sqrt, pi, ...
    env.update({
        '+': op.add, '-': op.sub, '*': op.mul, '/': op.truediv,
        '>': op.gt, '<': op.lt, '>=': op.ge, '<=': op.le, '=': op.eq,
        'abs': abs,
        'append': op.add,
        'apply': lambda proc, args: proc(*args),
        'begin': lambda *x: x[-1],
        'car': lambda x: x[0],
        'cdr': lambda x: x[1:],
        'cons': lambda x, y: [x] + y,
        'eq?': op.is_,
        'expt': pow,
        'equal?': op.eq,
        'length': len,
        'list': lambda *x: List(x),
        'list?': lambda x: isinstance(x, List),
        'map': map,
        'max': max,
        'min': min,
        'not': op.not_,
        'null?': lambda x: x == [],
        'number?': lambda x: isinstance(x, Number),
        'print': print,
        'procedure?': callable,
        'round': round,
        'symbol?': lambda x: isinstance(x, Symbol),
    })
    return env

global_env = standard_env()
```

Evaluation: eval

We are now ready for the implementation of `eval`. As a refresher, we repeat the table of Lispy Calculator forms:

Expression	Syntax	Semantics and Example
variable reference	<i>symbol</i>	A symbol is interpreted as a variable name; its value is the variable's value. Example: $r \Rightarrow 10$ (assuming r was previously defined to be 10)
constant literal	<i>number</i>	A number evaluates to itself. Examples: $12 \Rightarrow 12$ or $-3.45e+6 \Rightarrow -3.45e+6$
conditional	(if test conseq alt)	Evaluate <i>test</i> ; if true, evaluate and return <i>conseq</i> ; otherwise <i>alt</i> . Example: (if (> 10 20) (+ 1 1) (+ 3 3)) $\Rightarrow 6$

definition	(define <i>symbol exp</i>)	Define a new variable and give it the value of evaluating the expression <i>exp</i> . Examples: (define r 10)
procedure call	(<i>proc arg...</i>)	If <i>proc</i> is anything other than one of the symbols if, define, or quote then it is treated as a procedure. Evaluate <i>proc</i> and all the <i>args</i> , and then the procedure is applied to the list of <i>arg</i> values. Example: (sqrt (* 2 8)) ⇒ 4.0

Here is the code for eval, which closely follows the table:

```
def eval(x: Exp, env=global_env) -> Exp:
    "Evaluate an expression in an environment."
    if isinstance(x, Symbol):          # variable reference
        return env[x]
    elif isinstance(x, Number):        # constant number
        return x
    elif x[0] == 'if':                 # conditional
        (_, test, conseq, alt) = x
        exp = (conseq if eval(test, env) else alt)
        return eval(exp, env)
    elif x[0] == 'define':             # definition
        (_, symbol, exp) = x
        env[symbol] = eval(exp, env)
    else:                              # procedure call
        proc = eval(x[0], env)
        args = [eval(arg, env) for arg in x[1:]]
        return proc(*args)
```

We're done! You can see it all in action:

```
>>> eval(parse("(begin (define r 10) (* pi (* r r))"))
314.1592653589793
```

Interaction: A REPL

It is tedious to have to enter eval(parse("...")) all the time. One of Lisp's great legacies is the notion of an interactive read-eval-print loop: a way for a programmer to enter an expression, and see it immediately read, evaluated, and printed, without having to go through a lengthy build/compile/run cycle. So let's define the function repl (which stands for read-eval-print-loop), and the function schemestr which returns a string representing a Scheme object.

```
def repl(prompt='lis.py> '):
    "A prompt-read-eval-print loop."
    while True:
        val = eval(parse(raw_input(prompt)))
        if val is not None:
            print(schemestr(val))

def schemestr(exp):
    "Convert a Python object back into a Scheme-readable string."
    if isinstance(exp, List):
        return '(' + ' '.join(map(schemestr, exp)) + ')'
    else:
        return str(exp)
```

Here is repl in action:

```
>>> repl()
lis.py> (define r 10)
lis.py> (* pi (* r r))
314.159265359
lis.py> (if (> (* 11 11) 120) (* 7 6) oops)
42
lis.py> (list (+ 1 1) (+ 2 2) (* 2 3) (expt 2 3))
lis.py>
```

Language 2: Full Lispy

We will now extend our language with three new special forms, giving us a much more nearly-complete Scheme subset:

Expression	Syntax	Semantics and Example
quotation	(quote <i>exp</i>)	Return the <i>exp</i> literally; do not evaluate it. Example: (quote (+ 1 2)) ⇒ (+ 1 2)
assignment	(set! <i>symbol exp</i>)	Evaluate <i>exp</i> and assign that value to <i>symbol</i> , which must have been previously defined (with a define or as a parameter to an enclosing procedure). Example: (set! r2 (* r r))
procedure	(lambda (<i>symbol...</i>) <i>exp</i>)	Create a procedure with parameter(s) named <i>symbol...</i> and <i>exp</i> as the body. Example: (lambda (r) (* pi (* r r)))

The lambda special form (an obscure nomenclature choice that refers to Alonzo Church's [lambda calculus](#)) creates a procedure. We want procedures to work like this:

```
lis.py> (define circle-area (lambda (r) (* pi (* r r))))
lis.py> (circle-area (+ 5 5))
314.159265359
```

There are two steps here. In the first step, the lambda expression is evaluated to create a procedure, one which refers to the global variables pi and *, takes a single parameter, which it calls r. This procedure is used as the value of the new variable circle-area. In the second step, the procedure we just defined is the value of circle-area, so it is called, with the value 10 as the argument. We want r to take on the value 10, but it wouldn't do to just set r to 10 in the global environment. What if we were using r for some other purpose? We wouldn't want a call to circle-area to alter that value. Instead, we want to arrange for there to be a *local* variable named r that we can set to 10 without worrying about interfering with any global variable that happens to have the same name. The process for calling a procedure introduces these new local variable(s), binding each symbol in the parameter list of the function to the corresponding value in the argument list of the function call.

Redefining Env as a Class

To handle local variables, we will redefine Env to be a subclass of dict. When we evaluate (circle-area (+ 5 5)), we will fetch the procedure body, (* pi (* r r)), and evaluate it in an environment that has r as the sole local variable (with value 10), but also has the global environment as the "outer" environment; it is there that we will find the values of * and pi. In other words, we want an environment that looks like this, with the local (blue) environment nested inside the outer (red) global environment:

```
pi: 3.141592653589793
*: <built-in function mul>
...
r: 10
```

When we look up a variable in such a nested environment, we look first at the innermost level, but if we don't find the variable name there, we move to the next outer level. Procedures and environments are intertwined, so let's define them together:

```
class Env(dict):
    "An environment: a dict of {'var': val} pairs, with an outer Env."
    def __init__(self, parms=(), args=(), outer=None):
        self.update(zip(parms, args))
        self.outer = outer
    def find(self, var):
        "Find the innermost Env where var appears."
        return self if (var in self) else self.outer.find(var)
```

```

class Procedure(object):
    "A user-defined Scheme procedure."
    def __init__(self, parms, body, env):
        self.parms, self.body, self.env = parms, body, env
    def __call__(self, *args):
        return eval(self.body, Env(self.parms, args, self.env))

global_env = standard_env()

```

We see that every procedure has three components: a list of parameter names, a body expression, and an environment that tells us what other variables are accessible from the body. For a procedure defined at the top level this will be the global environment, but it is also possible for a procedure to refer to the local variables of the environment in which it was *defined* (and not the environment in which it is *called*).

An environment is a subclass of dict, so it has all the methods that dict has. In addition there are two methods: the constructor `__init__` builds a new environment by taking a list of parameter names and a corresponding list of argument values, and creating a new environment that has those {variable: value} pairs as the inner part, and also refers to the given outer environment. The method `find` is used to find the right environment for a variable: either the inner one or an outer one.

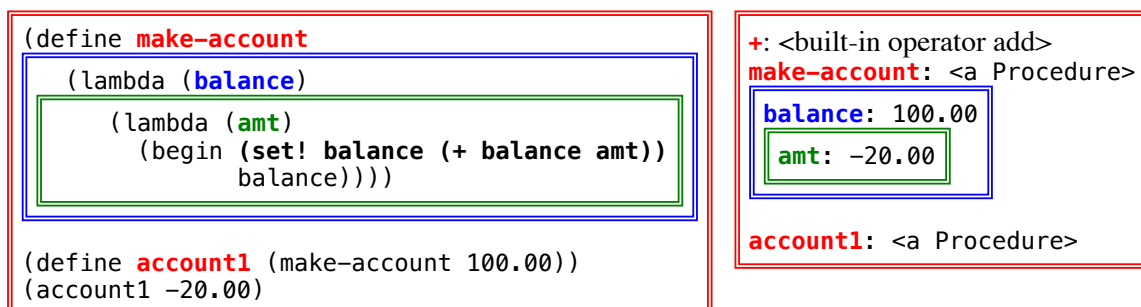
To see how these all go together, here is the new definition of `eval`. Note that the clause for variable reference has changed: we now have to call `env.find(x)` to find at what level the variable `x` exists; then we can fetch the value of `x` from that level. (The clause for `define` has not changed, because a `define` always adds a new variable to the innermost environment.) There are two new clauses: for `set!`, we find the environment level where the variable exists and set it to a new value. With `lambda`, we create a new procedure object with the given parameter list, body, and environment.

```

def eval(x, env=global_env):
    "Evaluate an expression in an environment."
    if isinstance(x, Symbol):      # variable reference
        return env.find(x)[x]
    elif not isinstance(x, List): # constant
        return x
    op, *args = x
    if op == 'quote':              # quotation
        return args[0]
    elif op == 'if':              # conditional
        (test, consequ, alt) = args
        exp = (consequ if eval(test, env) else alt)
        return eval(exp, env)
    elif op == 'define':          # definition
        (symbol, exp) = args
        env[symbol] = eval(exp, env)
    elif op == 'set!':            # assignment
        (symbol, exp) = args
        env.find(symbol)[symbol] = eval(exp, env)
    elif op == 'lambda':          # procedure
        (parms, body) = args
        return Procedure(parms, body, env)
    else:                         # procedure call
        proc = eval(op, env)
        vals = [eval(arg, env) for arg in args]
        return proc(*vals)

```

To appreciate how procedures and environments work together, consider this program and the environment that gets formed when we evaluate `(account1 -20.00)`:



Each rectangular box represents an environment, and the color of the box matches the color of the variables that are newly defined in the environment. In the last two lines of the program we define `account1` and call `(account1 -20.00)`; this represents the creation of a bank account with a 100 dollar opening balance, followed by a 20 dollar withdrawal. In the process of evaluating `(account1 -20.00)`, we will eval the expression highlighted in yellow. There are three variables in that expression. `amt` can be found immediately in the innermost (green) environment. But `balance` is not defined there: we have to look at the green environment's outer env, the blue one. And finally, the variable `+` is not found in either of those; we need to do one more outer step, to the global (red) environment. This process of looking first in inner environments and then in outer ones is called *lexical scoping*. `Env.find(var)` finds the right environment according to lexical scoping rules.

Let's see what we can do now:

```
>>> repl()
lis.py> (define circle-area (lambda (r) (* pi (* r r))))
lis.py> (circle-area 3)
28.274333877
lis.py> (define fact (lambda (n) (if (<= n 1) 1 (* n (fact (- n 1))))))
lis.py> (fact 10)
3628800
lis.py> (fact 100)
9332621544394415268169923885626670049071596826438162146859296389521759999322991
560894146397615651828625369792082722375825118521091686400000000000000000000000
lis.py> (circle-area (fact 10))
4.1369087198e+13
lis.py> (define first car)
lis.py> (define rest cdr)
lis.py> (define count (lambda (item L) (if L (+ (equal? item (first L)) (count item (rest L))) 0)))
lis.py> (count 0 (list 0 1 2 3 0 0))
3
lis.py> (count (quote the) (quote (the more the merrier the bigger the better)))
4
lis.py> (define twice (lambda (x) (* 2 x)))
lis.py> (twice 5)
10
lis.py> (define repeat (lambda (f) (lambda (x) (f (f x)))))
lis.py> ((repeat twice) 10)
40
lis.py> ((repeat (repeat twice)) 10)
160
lis.py> ((repeat (repeat (repeat twice))) 10)
2560
lis.py> ((repeat (repeat (repeat (repeat twice)))) 10)
655360
lis.py> (pow 2 16)
65536.0
lis.py> (define fib (lambda (n) (if (< n 2) 1 (+ (fib (- n 1)) (fib (- n 2))))))
lis.py> (define range (lambda (a b) (if (= a b) (quote ()) (cons a (range (+ a 1) b)))))
lis.py> (range 0 10)
(0 1 2 3 4 5 6 7 8 9)
lis.py> (map fib (range 0 10))
(1 1 2 3 5 8 13 21 34 55)
lis.py> (map fib (range 0 20))
(1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765)
```

We now have a language with procedures, variables, conditionals (`if`), and sequential execution (the `begin` procedure). If you are familiar with other languages, you might think that a `while` or `for` loop would be needed, but Scheme manages to do without these just fine. The Scheme report says "Scheme demonstrates that a very small number of rules for forming expressions, with no restrictions on how they are composed, suffice to form a practical and efficient programming language." In Scheme you iterate by defining recursive functions.

How Small/Fast/Complete/Good is Lispy?

In which we judge Lispy on several criteria:

- **Small:** Lispy is *very* small: 117 non-comment non-blank lines; 4K of source code. (An earlier version was just 90 lines, but had fewer standard procedures and was perhaps a bit too terse.) The smallest version of my Scheme in Java, [Jscheme](#), was 1664 lines and 57K of source. Jscheme was originally called SILK (Scheme in Fifty Kilobytes), but I only kept under that limit by counting bytecode rather than source code. Lispy does much better; I think it meets Alan Kay's 1972 [claim](#) that *you could define the "most powerful language in the world" in "a page of code."* (However, I think Alan would disagree, because he would count the Python compiler as part of the code, putting me *well* over a page.)

```
bash$ grep "^\\s*[^#\\s]" lis.py | wc
      117      497     4276
```

- **Fast:** Lispy computes (fact 100) exactly in 0.003 seconds. That's fast enough for me (although far slower than most other ways of computing it).
- **Complete:** Lispy is not very complete compared to the Scheme standard. Some major shortcomings:
 - **Syntax:** Missing comments, quote and quasiquote notation, # literals, the derived expression types (such as cond, derived from if, or let, derived from lambda), and dotted list notation.
 - **Semantics:** Missing call/cc and tail recursion.
 - **Data Types:** Missing strings, characters, booleans, ports, vectors, exact/inexact numbers. Python lists are actually closer to Scheme vectors than to the Scheme pairs and lists that we implement with them.
 - **Procedures:** Missing over 100 primitive procedures.
 - **Error recovery:** Lispy does not attempt to detect, reasonably report, or recover from errors. Lispy expects the programmer to be perfect.
- **Good:** That's up to the readers to decide. I found it was good for my purpose of explaining Lisp interpreters.

True Story

To back up the idea that it can be very helpful to know how interpreters work, here's a story. Way back in 1984 I was writing a Ph.D. thesis. This was before LaTeX, before Microsoft Word for Windows—we used troff. Unfortunately, troff had no facility for forward references to symbolic labels: I wanted to be able to write "As we will see on page @theorem-x" and then write something like "@(set theorem-x \n%)" in the appropriate place (the troff register \n% holds the page number). My fellow grad student Tony DeRose felt the same need, and together we sketched out a simple Lisp program that would handle this as a preprocessor. However, it turned out that the Lisp we had at the time was good at reading Lisp expressions, but so slow at reading character-at-a-time non-Lisp expressions that our program was annoying to use.

From there Tony and I split paths. He reasoned that the hard part was the interpreter for expressions; he needed Lisp for that, but he knew how to write a tiny C routine for reading and echoing the non-Lisp characters and link it in to the Lisp program. I didn't know how to do that linking, but I reasoned that writing an interpreter for this trivial language (all it had was set variable, fetch variable, and string concatenate) was easy, so I wrote an interpreter in C. So, ironically, Tony wrote a Lisp program (with one small routine in C) because he was a C programmer, and I wrote a C program because I was a Lisp programmer.

In the end, we both got our theses done ([Tony](#), [Peter](#)).

The Whole Thing

The whole program is here: [lis.py](#).

Further Reading

To learn more about Scheme consult some of the fine books (by [Friedman and Fellesein](#), [Dybvig](#), [Queinnec](#), [Harvey and Wright](#) or [Sussman and Abelson](#)), videos (by [Abelson and Sussman](#)), tutorials (by [Dorai](#), [PLT](#), or [Neller](#)), or the [reference manual](#).

I also have another page describing a [more advanced version of Lispy](#).
